



Fig. 6: CUDA programming model

programming model and its popularity render a large number of existing applications available to FPGA acceleration.

Even though CUDA is driven by the GPU computing domain, CUDA kernels can indeed be translated with FUDA into efficient, customised multi-core compute engines on the FPGA.

CUDA programming

CUDA enables general-purpose computing on the GPU (GPGPU) through a C-like API which is gaining considerable popularity. The CUDA programming model exposes parallelism through a data-parallel SPMD kernel function. Each kernel implicitly describes multiple CUDA threads that are organised in groups called thread-blocks. Thread-blocks are further organised into a grid structure (Fig. 6).

Threads within a thread block are executed by the streaming processors of a single GPU streaming multiprocessor and allowed to synchronise and share data through the streaming multiprocessor's shared memory. On the other hand, synchronisation of thread-blocks is not supported.

Thread-block threads are launched in SIMD bundles called 'warps'. Warps consisting of threads with highly diverse control flow

result in low performance execution. Thus, for successful GPU acceleration it is critical that threads are organised in warps based on their control flow characteristics.

The CUDA memory model leverages separate memory spaces with diverse characteristics. Shared memory refers to on-chip SRAM blocks, with each block being accessible by a single streaming multiprocessor (Fig. 6). Global memory, on the other hand, is the off-chip DRAM that is accessible by all streaming multiprocessors. Shared memory is fast but small, whereas global memory is long-latency but abundant. There are also two read-only off-chip memory spaces, constant and texture, which are cached and provide special features for kernels executed on the GPU.

FASTCUDA

FASTCUDA platform provides the necessary software toolset, hardware architecture and design methodology to efficiently adapt the CUDA approach into a new FPGA design flow. With FASTCUDA, CUDA kernels of a CUDA-based application are partitioned into two groups with minimal user intervention: those that are compiled and executed in parallel software, and those that are synthesised and implemented in hardware. An advanced low-power

FPGA can provide the processing power (via numerous embedded micro-CPUs) and logic capacity for both software and hardware implementations of CUDA kernels.

FASTCUDA approach

Today's complex systems employ both software and hardware implementations of components. General-purpose CPUs, or more specialised processors such as GPUs, running the software components, will routinely interact with special-purpose ASICs or FPGAs that implement time-critical functions in hardware. In these systems, separation of duties between software and hardware is usually very clear. FASTCUDA aims to bring software and hardware closer together, interacting and cooperating for execution of a common source code. As a proof of concept, FASTCUDA focuses on source codes written in CUDA.

Source code example follows:

```
//kernel
__global__ void vectorAdd (float *A,
                           float *B, float *C) {
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

# define N 100
#define M N*sizeof (int)

//host program
main() {
    int A[M], B[M], C[M];
    ...

    //copy input vectors from host memory
    to device memory
    cudaMemcpy( d_A, A, M,
               cudaMemcpyHostToDevice);

    cudaMemcpy( d_B, B, M,
               cudaMemcpyHostToDevice);

    // kernel invocation
    vectorAdd<<1,N>>>(d_A, d_B, d_C);
    //copy output vectors from device
    memory to host memory
    cudaMemcpy(C, d_C, M,
               cudaMemcpyDeviceToHost );
    ...
}
```